

Prism Core

Engine Guide — how a raw event becomes adaptive audio, what you must know before changing it, and how to embed it in your software

Repository	prism-core (private)
Documents the code at	commit 2f55a27, branch claude/repo-context-overview-8xbj1j
Public surface described	C ABI include/prism/prism_core.h, version 0.3.0 (20 exported functions)
Audience	Engineers integrating, extending or reviewing the engine
Status of the numbers	Every value in Part II was produced by running this commit's code, not by re-deriving formulas
Date	19 September 2026

Scope. This guide covers the engine that exists today: the Prism Context Engine (PCE), the Prism State Vector (PSV) contract, and the Prism Generative Audio Engine (PGAE), behind one C ABI. The proposed haptics engine (PGHE) is specified separately in docs/prism-pghe-consumption-spec.md and is **not** implemented; it is out of scope here except where the architecture anticipates it.

Confidentiality. This repository is private and nothing in it is disclosed until the patent position allows. Treat this document the same way.

Contents

Part	What it answers	Page topic
Part I	What goes in, what comes out, and every step between	The data flow, stage by stage
Part II	How the output changes when the input changes	Measured transfer tables from the real engine
Part III	What must I know before I change anything?	Invariants, constant map, tripwires, safe-change recipe
Part IV	How do I attach this as a module to my software?	ABI, lifecycle, threading, inputs, outputs, code
Appendix A	Every ABI function at a glance	Function reference
Appendix B	Every tunable constant and where it lives	Constant reference
Appendix C	Terms	Glossary

The engine in one paragraph

Prism Core turns cheap, privacy-safe signals about what a person is doing into continuously adapting ambient audio. Three kinds of raw event go in (window switches, idle durations, task deadlines) plus the clock. An inference engine fuses them into a four-dimensional **Prism State Vector** — arousal, valence, cognitive load, readiness — where every dimension carries its own confidence. A separate audio engine consumes only that vector and renders a continuous soundscape from pre-loaded loops. Nothing leaves the device, and the two engines cannot see each other: the state vector is the only thing that crosses between them.

The whole pipeline on one line

```
host events  -> PCE  -> [ Prism State Vector ]  -> PGAE  -> float32 audio
(switch/idle/   (3 adapters   (4 dims, each   (mapping   (mono, your
deadlines,      + fusion)      value+confidence) + DSP)      sample rate)
+ clock)        |              |
                  |              +-> prism_get_psv  (read-out for your UI)
                  +-- emits every 30 s, or sooner on a >= 0.1 change
```

The one structural rule. The PCE emits a state vector and nothing else — never an audio parameter. The PGAE reads a state vector and nothing else — never an input event. This 'firewall' is enforced by the build graph and by a test, not by convention. Everything else in this document follows from it.

Part I — From input to output, stage by stage

There are eight stages. Stages 1–4 run on an inference thread the library owns. Stages 6–8 run on an audio thread you own (or one the library opens for you). Stage 5 is the handoff between them, and it is the only place the two halves touch.

Stage 0 — What you actually feed in

The engine accepts three event types and a clock. That is the entire input surface. Note what is not there: no app names, no window titles, no keystrokes, no screenshots, no microphone, no camera, no network. A window switch is a bare timestamp; idle is a duration; a task is a due instant plus a priority. App identity is discarded in your shell before it reaches the ABI.

Input	ABI call	What it means	Who calls it
Window / app switch	<code>prism_report_app_switch(core, t_ms)</code>	The user moved from one context to another at this instant. Frequency of these drives the scatter signal.	Your shell, on every focus change
Idle sample	<code>prism_report_idle(core, t_ms, idle_ms)</code>	As of this instant, the user has been idle for <code>idle_ms</code> . This is a heartbeat : send one on a fixed timer, not only when idle.	Your shell, every ~5 s
Task deadlines	<code>prism_report_task_deadlines(core, tasks, count)</code>	The complete current task list as due instants plus priorities (0 low, 1 medium, 2 high). Replaces the previous snapshot entirely.	Your shell, whenever the list changes
Clock + timezone	<code>prism_config.tz_offset_min</code>	Supplied once at creation (UTC minus local, in minutes). Drives the circadian prior; the engine reads the system clock itself.	Set at <code>prism_create</code>

The idle heartbeat is load-bearing. Behavioural confidence is derived from how many idle samples arrived in the last two minutes, not from their content. If you only report idle when the user goes away, coverage stays near zero, behavioural confidence stays near zero, and the behavioural half of the vector is inert. Send the heartbeat on a timer.

Stage 1 — Ingress and the rolling window

Events land in memory and nowhere else. Switch timestamps and idle samples go into a rolling **120-second window**; anything older than `now - 120 s` is evicted on the next read. The task list is a snapshot that replaces its predecessor. Nothing is written to disk, ever — that is a privacy invariant of the repository, not a configuration choice.

Ingress calls are thread-safe and cheap. They take a lock shared with the inference thread and append to a vector; they never block on I/O. You may call them from any thread except an audio callback.

Stage 2 — Three adapters reduce the raw signals

Each adapter reduces its signal to a **value in [0,1] plus a confidence in [0,1]**. An adapter knows nothing about the state vector and nothing about audio. All three are pure functions of their inputs and the clock.

2a. Deadline proximity — pce/src/deadline.cpp

```
pressure = max over tasks of: priority_weight x exp(-max(0, due_ms - now) / tau)
tau = 48 h           weights: low 0.4, medium 0.7, high 1.0
confidence = 0.9 if the task list is non-empty, else 0.0 (value falls back to 0.5)
```

The single most pressing task drives the signal; overdue tasks clamp at maximum proximity. Confidence reflects availability, not freshness — a due date is a fact, so it is trusted at 0.9 as soon as it exists, and not at all when the list is empty. Measured, for a high-priority task:

Due in	0 h	2 h	12 h	24 h	48 h	168 h (1 wk)
pressure	1.000	0.959	0.779	0.607	0.368	0.030

2b. Attention / behaviour — pce/src/attention.cpp

```
switches_per_min = switches in window / 2           (window is 120 s)
scatter    = 1 - exp(-switches_per_min / 6)          0 = settled, 1 = fragmented
activity   = 1 - (idle samples over 15 s) / (all idle samples in window)
coverage   = min(1, idle samples / 24)               24 = a full window at 5 s spacing
confidence = coverage x 0.7
```

Two distinct drivers come out: **activity** (present versus away) and **scatter** (settled versus context-switching). They push different dimensions, which is why they are kept separate. The 0.7 ceiling exists because behaviour is a noisier proxy than a calendar date, so even a perfectly covered window never reaches full confidence.

2c. Circadian phase — pce/src/circadian.cpp

```
daily      = 0.5 + 0.4 x cos(2 pi (h - 16) / 24)      h = local hour, fractional
post_lunch = 0.1 x exp(-(h - 14)^2 / (2 x 1.5^2))
alertness  = clamp01(daily - post_lunch)             confidence = 0.5, always
```

A 24-hour cosine peaking at 16:00 and troughing near 04:00, minus a small post-lunch dip centred on 14:00. It is a population-level prior rather than a measurement of this person, so it ships at a fixed middling confidence of 0.5. Measured across the day:

Local hour	00	04	08	10	12	14	16	18	20	22
alertness	0.300	0.100	0.300	0.497	0.659	0.746	0.859	0.844	0.700	0.500

Note the dip: 14:00 (0.746) sits below the smooth curve, and the peak is at 16:00, not midday.

Stage 3 — Fusion into the state vector

Fusion is one small function, pce/src/fusion.cpp. Each PSV dimension is a confidence-weighted blend of several contributions. A contribution is a target value, the confidence in it, and an **importance** — a fixed weight saying how much that signal should matter for that dimension.

```
value      = sum(confidence x importance x target) / sum(confidence x importance)
confidence = sum(confidence x importance) / sum(importance)
if no contribution has any confidence: value = 0.5, confidence = 0.0
```

Read the second line carefully, because it is the subtle one: a dimension's confidence is the fraction of its expected input that is actually present and trusted. If a dimension draws on three signals and only one is available, its confidence is low even if that one signal is certain.

Dimension	Contributions (target ← signal, importance)	Meaning
arousal	deadline pressure (1.0) • circadian alertness (0.7) • behavioural arousal (0.7) where behavioural arousal = activity × (0.55 + 0.45 × scatter)	Activation. Deadlines key you up, circadian sets the daily baseline, behaviour adds the moment. Being away suppresses it regardless of scatter.
cognitive_load	deadline pressure (1.0) • scatter (0.9)	Pressure and fragmentation both raise load. Deliberate property: high pressure with low scatter lands at moderate load (flow), while scattered-under-pressure lands high.
readiness	circadian alertness (1.0) • (1 - deadline pressure) (0.4)	Capacity. The daily curve, eroded by sustained pressure.
valence	none — hard-coded {0.5, 0.0}	Reserved slot. Inert in v1: it is emitted, but at zero confidence, so no consumer acts on it.

Stage 4 — When a vector is emitted

An inference tick runs every **5 seconds** by default. It does not emit every tick. A vector is published when either condition holds:

- the nominal cadence has elapsed — **30 s** by default for the Aqademiq profile, or
- any dimension's value moved by at least **0.1** since the last emission (the 'significant change' rule).

Both numbers are configurable at creation (cadence_ms, check_interval_ms, significant_delta). Every emission carries a strictly monotonic sequence, safe to deduplicate on, and an update_timestamp_ms. The very first vector after prism_start is the **neutral vector**: every value 0.5, every confidence 0.0.

Stage 5 — The handoff to the audio thread

The vector crosses from the inference thread to the audio thread through a pre-allocated, wait-free **triple buffer** (psv/include/psv/exchange.h). One writer, one reader, no mutex, no allocation, no tearing: the reader always gets the newest complete snapshot or nothing. The audio thread polls it exactly once per callback.

What crosses is RtStateVector — a 112-byte trivially copyable mirror of the vector with fixed arrays instead of strings, because copying a std::string can allocate and allocation is forbidden on the audio thread.

Stage 6 — Confidence weighting (the consumer's first act)

Before any mapping happens, every dimension is collapsed to a single **effective value** that folds in its own confidence:

```
effective(d) = 0.5 + (value(d) - 0.5) × confidence(d)
```

At confidence 0 the dimension is exactly neutral no matter how extreme its value; at confidence 1 it passes through untouched. This is per-dimension and strictly linear — there is no threshold anywhere. It is the mechanism that makes an uncertain estimate quietly stop steering the output instead of steering it wrongly, and it is the single most important line in the engine.

Stage 7 — The mapping law: state to audio parameters

One pure function, `pgae/src/mapping.cpp`, is the only place a state vector becomes audio intent. It takes the three active effective values, centres them on neutral, and computes every audio parameter as a saturated affine function of those deviations.

```
a = effective(arousal) - 0.5      l = effective(cognitive_load) - 0.5
r = effective(readiness) - 0.5    (valence has no term in v1)
brightness = clamp01(0.55 + 0.9a - 0.8l)    cutoff_hz = 300 x 40^brightness    (300 Hz .. 12 kHz)
density     = clamp01(0.5 + 0.9a - 1.0l)
gain[bed]   = clamp01(0.6 + 0.6l)    steady bed rises under load - something calm to hold
gain[sub]   = clamp01(0.5 - 0.6r)    low grounding rises as readiness falls
gain[pulse] = clamp01(0.5 + 0.7a - 0.6l)
gain[lead]  = clamp01(0.5 + 0.4a - 0.9l)    melodic foreground recedes hardest under load
gain[air]   = clamp01(0.5 + 0.5a - 0.6l)
active[bed] = active[sub] = always on
active[pulse] = density >= 0.35    active[air] = density >= 0.55    active[lead] = density >= 0.72
```

Two things are worth internalising here. First, **load is the dominant term** and it points the opposite way from arousal on nearly every parameter: as cognitive load rises the filter closes, layers drop out, and the melodic foreground recedes fastest. That is the focus-protection principle, expressed as coefficient signs rather than as a rule. Second, the **density gates** are thresholds on a mapped parameter, never on confidence — the confidence path stays linear and threshold-free.

Stage 8 — The render path

The mapped parameters are targets, not values. Everything the listener hears is smoothed, scheduled and rail-bounded on the way out. Per sample, in order:

1. **Smoothed cutoff.** A one-pole approach to the target with a 0.6 s time constant; the biquad coefficients refresh every 32 samples, but the parameter itself moves every sample so it can never step.
2. **Per-stem render.** Each stem reads its loop at phase `% loop_frames` (sample-accurate), multiplied by its density envelope and its smoothed level (0.25 s time constant). Level in `[0,1]` becomes amplitude as $0.4 \times g^{1.5}$.
3. **Density crossfades.** When a gate flips, the change is scheduled at that stem's next loop boundary and applied as a 1.5 s equal-power fade — never applied immediately, never stepped. A re-scheduled fade ramps from the envelope's current value, so rapid flips cannot click.
4. **Scene crossfade (optional).** If a scene swap is in flight, two decks render and sum under equal-power gains. This is the only place two scenes exist at once; the outgoing scene is handed back to the control thread for release, never freed on the audio thread.
5. **Master low-pass** at the smoothed cutoff, then a master gain of 0.7 with a 0.5 s linear fade-in from exact silence at start.
6. **Limiter, last and unbypassable.** Peak limiter at -3 dBFS (3 ms attack, 100 ms release) plus a hard clamp behind it. Nothing upstream can exceed the ceiling.

Real-time rules, in one sentence. The render path never allocates, never locks, never logs, never blocks and never touches a file. Everything it reads was allocated when the scene loaded. Two automated checks enforce this: an instrumented-allocator test over a ten-minute render, and a static audit of the compiler-computed include closure of the render path for any locking, blocking, logging or allocator symbol.

What comes out

Output	How you get it	Shape
Adaptive audio	<code>prism_render(core, float*, frames)</code> from your audio callback, or let the library open a device with <code>prism_device_start</code>	Mono float32, at the scene's sample rate (<code>prism_sample_rate</code>)
Current state	<code>prism_get_psv(core, &out)</code> from any thread, poll it on a UI timer	4 values + 4 confidences, timestamp, sequence, mode hint

Part I on one page

#	Stage	Where it lives	Thread	What leaves it
0	Event ingress	<code>prism_core.cpp</code> (ABI)	any	Timestamps, durations, due instants
1	Rolling 120 s window	<code>pce/src/attention.cpp</code>	inference	The events of the last two minutes
2	Three adapters	<code>pce/src/{deadline,attention,circadian}.cpp</code>	inference	value + confidence, per signal
3	Fusion	<code>pce/src/fusion.cpp</code>	inference	A Prism State Vector (4 x value+confidence)
4	Emission policy	<code>pce/src/pce.cpp</code>	inference	A vector, every 30 s or on a 0.1 change
5	Wait-free handoff	<code>psv/include/psv/exchange.h</code>	crosses	A 112-byte snapshot, no locks
6	Confidence weighting	<code>psv/include/psv/psv.h</code>	audio	One effective value per dimension
7	Mapping law	<code>pgae/src/mapping.cpp</code>	audio	Cutoff, density, 5 gains, 5 gates
8	Render + rails	<code>pgae/src/engine.cpp</code>	audio	float32 samples, limiter-bounded

Read down the last column and the firewall is visible as a shape: raw events never travel past stage 3, and audio parameters never exist before stage 7. The only thing that crosses the middle is the vector.

Part II — How the output changes with the input

Every number in this part was produced by compiling this commit and running scripted inputs through the real PCE and the real mapping function. Nothing here is a re-derivation by hand. Reading guide: a/l/r are arousal, cognitive load and readiness as value/confidence; eff is the confidence-weighted effective value that actually drives audio; cut is the master low-pass cutoff; den is density; layers listed are those above their gate.

A. Nine end-to-end scenarios

Scenario	arousal	load	readiness	cutoff Hz	density	layers on
Cold start (no input yet)	0.50/0.00	0.50/0.00	0.50/0.00	2282	0.500	bed, sub, pulse
Deep focus, deadline 24 h	0.57/0.72	0.36/0.81	0.45/0.61	3783	0.660	bed, sub, pulse, air
Deep focus, deadline 2 h	0.75/0.72	0.56/0.81	0.31/0.61	3584	0.612	bed, sub, pulse, air
Scattered, deadline 2 h	0.92/0.72	0.92/0.81	0.49/0.61	2325	0.437	bed, sub, pulse
Scattered, no deadline	0.88/0.35	0.87/0.33	0.81/0.36	2494	0.500	bed, sub, pulse
Away (idle 20 s)	0.48/0.72	0.36/0.81	0.63/0.61	3026	0.600	bed, sub, pulse, air
Night wind-down (23:00)	0.30/0.72	0.12/0.81	0.61/0.61	3516	0.679	bed, sub, pulse, air
Fresh morning, no tasks	0.53/0.35	0.15/0.33	0.40/0.36	3301	0.623	bed, sub, pulse, air
Post-lunch dip (14:00)	0.56/0.72	0.37/0.81	0.68/0.61	3626	0.647	bed, sub, pulse, air

The headline comparison is rows 3 and 4. Same deadline two hours out, same time of day; the only difference is that the user is switching windows constantly. Load goes from 0.56 to 0.92, the filter closes from 3584 Hz to 2325 Hz, density falls below the air gate, and the airy layer drops out. Under pressure and fragmentation the soundscape gets darker and simpler — it recedes rather than energising. That is the entire product thesis, visible in four numbers.

B. One input at a time

B1. Deadline distance — high priority, 10:00, settled, full behavioural coverage

Due in	arousal	load	readiness	cutoff Hz	density	layers on
1 h	0.761	0.576	0.298	3573	0.609	bed sub pulse air
6 h	0.711	0.519	0.338	3626	0.623	bed sub pulse air
12 h	0.658	0.458	0.382	3684	0.637	bed sub pulse air
24 h	0.569	0.357	0.454	3783	0.660	bed sub pulse air
48 h	0.445	0.216	0.554	3923	0.693	bed sub pulse air
96 h	0.325	0.080	0.651	4065	0.724	bed sub pulse lead air
1 week	0.271	0.018	0.695	4131	0.739	bed sub pulse lead air

Monotone in both directions: pressure raises arousal and load, and lowers readiness. Watch the lead layer — the melodic foreground — appear only once density crosses 0.72, which happens when the deadline is four days out. Melody is a reward for having room to think.

B2. Window-switch rate — deadline 24 h high, 10:00

Switches per 2 min	arousal	load	readiness	cutoff Hz	density	layers on
0	0.569	0.357	0.454	3783	0.660	bed sub pulse air
2	0.588	0.420	0.454	3411	0.622	bed sub pulse air
6	0.618	0.519	0.454	2902	0.562	bed sub pulse air
12	0.649	0.617	0.454	2471	0.503	bed sub pulse
24	0.678	0.713	0.454	2113	0.445	bed sub pulse
48	0.693	0.761	0.454	1953	0.416	bed sub pulse

Scatter saturates: the step from 0 to 6 switches costs more brightness than the step from 24 to 48, because $\text{scatter} = 1 - \exp(-\text{rate}/6)$ flattens out. Readiness does not move at all — it has no behavioural contribution.

B3. Time of day — deadline 24 h high, settled

Local hour	arousal	load	readiness	cutoff Hz	density
04:00	0.489	0.357	0.223	3121	0.608
09:00	0.548	0.357	0.395	3602	0.647
14:00	0.619	0.357	0.599	4268	0.693
16:00	0.641	0.357	0.664	4507	0.708
20:00	0.609	0.357	0.572	4173	0.687
23:00	0.548	0.357	0.395	3603	0.647

Load is flat because the circadian prior contributes nothing to it. Note that 09:00 and 23:00 are nearly identical — the curve is a cosine, so morning and late evening are symmetric about the 16:00 peak. If you want them to feel different, the circadian adapter is where you would change it, not the mapping.

B4. Idle duration reported on each heartbeat

Idle per sample	arousal	load	cutoff Hz	density	Effect
0 ms	0.569	0.357	3783	0.660	counts as active
10 000 ms	0.569	0.357	3783	0.660	still active — below threshold
16 000 ms	0.414	0.357	2605	0.559	counts as away
30 000 ms	0.414	0.357	2605	0.559	identical to 16 s

This one is a **step, not a ramp**. The 15-second threshold makes a sample either 'present' or 'away'; 10 s and 0 s are indistinguishable, and so are 16 s and 30 s. Only the fraction of away-samples in the window varies continuously. Integrators often expect idle to be proportional. It is not.

C. Confidence as an independent lever

Same extreme vector (arousal 0.9, load 0.9, readiness 0.2), confidence swept from 0 to 1:

Confidence	eff arousal	eff load	eff readiness	cutoff Hz	density	bed gain
0.00	0.500	0.500	0.500	2282	0.500	0.600
0.25	0.600	0.600	0.425	2367	0.490	0.660

Confidence	eff arousal	eff load	eff readiness	cutoff Hz	density	bed gain
0.50	0.700	0.700	0.350	2456	0.480	0.720
0.75	0.800	0.800	0.275	2549	0.470	0.780
1.00	0.900	0.900	0.200	2645	0.460	0.840

Every effective value moves exactly linearly with confidence, and at zero confidence the engine sits at precisely its neutral operating point (2282 Hz, density 0.5) regardless of how extreme the underlying estimate was. When you see the engine 'not responding', check confidence before you check values — a shell that never sends the idle heartbeat produces exactly this.

Five behaviours that surprise people

- **Low load opens the filter as much as high arousal does.** Night wind-down (arousal 0.30) is brighter than cold start, because load is near zero and the $-0.8l$ term dominates. Brightness is not a proxy for energy.
- **Confidence and value are independent axes.** 'Scattered, no deadline' has values nearly as extreme as 'scattered, deadline 2 h', but every confidence is roughly half, so the audio barely moves. An empty task list does not just remove deadline pressure — it lowers confidence on all three dimensions.
- **Arousal and load usually move together, then diverge.** Both rise with deadline pressure; only load responds to fragmentation. Deep pressure with no switching is the 'flow' corner: high arousal, moderate load, bright and dense.
- **Layers appear and disappear at loop boundaries, not instantly.** A gate crossing schedules a 1.5 s equal-power fade at that stem's next loop point, so the audible change can trail the state change by several seconds. This is deliberate; do not treat it as latency.
- **Valence does nothing.** It is emitted at confidence 0 and has no coefficient in the mapping. Reading it in your UI is fine; acting on it is not.

Part III — Changing the engine safely

This part assumes you are about to edit the repository. Read the four invariants first: they are the constraints that automated checks enforce, and breaking one fails the build or the test suite rather than producing a subtly wrong engine.

The four invariants

Invariant	What it means in practice	How it is enforced
The firewall	psv/ depends on nothing. pce/ and pgae/ each depend on psv/ and nothing else. pgae/ must never see pce/, or the reverse. Only core/ links both.	CMake target graph makes the headers unreachable; the <code>firewall_includes</code> ctest additionally greps for relative-path and macro-indirected includes that would evade it.
Real-time safety	No allocation, lock, log, blocking call or file I/O anywhere the audio thread can reach. Everything pre-allocated at scene load. Every gain and routing change ramped; nothing steps.	PgaeRtSafety runs an instrumented global allocator over a ten-minute render; <code>rt_lock_primitives</code> statically audits the compiler-computed include closure of the render path.
Probe parity	The PCE reproduces the validated TypeScript probe to 1e-6 per dimension per tick. The heuristic constants are dogfooding findings, not guesses — they are ported verbatim and must not be 'improved' in this slice.	<code>tests/golden/fixture-*.jsonl</code> replayed by the parity test; divergence fails.
Privacy	No network code anywhere in the core. Raw events are processed in memory and never written to disk. Only derived values may be logged, and only when the host opts in.	By construction — there is no networking library in the dependency set, and adding one is a stop-and-ask item.

Where every knob lives

If you want to change how the engine behaves, this table tells you which file to open and what you are risking. The right-hand column is the honest cost.

I want to change...	File	What it affects	What it costs
How fast deadline pressure ramps	<code>pce/include/pce/deadline.h</code>	tau (48 h), priority weights, base confidence 0.9	Breaks golden-trace parity. Requires re-recording fixtures and a documented decision.
How switching maps to scatter	<code>pce/include/pce/attention.h</code>	window 120 s, heartbeat 5 s, idle threshold 15 s, scale 6/min, confidence ceiling 0.7	Breaks parity. Also changes confidence, so it moves the whole vector, not just load.
The daily alertness curve	<code>pce/src/circadian.cpp</code>	peak hour, amplitude, post-lunch dip, fixed confidence 0.5	Breaks parity. This is the right place to fix morning/evening symmetry.
How signals combine into dimensions	<code>pce/src/fusion.cpp</code>	importances, the behavioural-arousal expression, which signals feed which dimension	Breaks parity, and changes confidence semantics. The highest-leverage and highest-risk file in the PCE.
How state becomes sound	<code>pgae/src/mapping.cpp</code>	brightness/density coefficients, per-stem gains, gate thresholds 0.35/0.55/0.72	Does not break PCE parity, but changes the validated feel. Re-run the render tests and dogfood.
Smoothing, crossfade, limiter	<code>pgae/include/pgae/engine.h</code> (PgaeOptions)	tc_gain 0.25 s, tc_cutoff 0.6 s, crossfade 1.5 s, amp_trim 0.4, master 0.7, limiter -3 dBFS	Risk of audible steps or clipping. The no-step and ceiling tests are your tripwire.

I want to change...	File	What it affects	What it costs
Emit cadence / responsiveness	prism_config at runtime	cadence_ms, check_interval_ms, significant_delta	Safe — these are host configuration, not engine constants. Start here.
The musical material	assets/scenes.json + stems	which loops make up a scene, their key and length	Safe for the engine; loop lengths should stay coprime so the composite does not repeat audibly.

The cheapest change is almost always the manifest or the config. Before touching a coefficient, ask whether a different scene, a different cadence, or a host-side mood override gets you there. Those paths carry no parity risk and no review burden.

The test suite is the contract

Run `ctest --preset debug` before and after any change. What each guard catches:

Check	Catches
firewall_includes	Any cross-module include that would collapse the architecture, including relative-path and macro-indirected attempts.
rt_lock_primitives	A mutex, logger, allocator or blocking call appearing anywhere in the render path's include closure — even indirectly, via a header you did not think about.
PgaeRtSafety	Any allocation on the audio thread during a ten-minute render with live state publishing.
PgaeRender.*	Limiter ceiling exceeded; a parameter stepping instead of gliding; a crossfade not landing exactly on the loop boundary; equal-power property violated.
PceParity / golden traces	Any divergence from the validated probe beyond 1e-6 per dimension per tick.
PsvSchema / JSON schema	An emitted vector that no longer validates against the published contract.
abi_test, c_compat.c	An ABI change that breaks the version tripwire or stops compiling as plain C (which would break the Dart bindings).

A safe-change recipe

1. Reproduce the current behaviour first. Run `ctest` green, and capture an offline render (`prism_harness --render before.wav --seconds 60`) so you have an A/B.
2. Make the smallest possible change, in one file, and write down why in a comment next to the constant. Every heuristic constant in this repo carries its rationale; yours must too.
3. Run the suite. If parity fails, stop and decide deliberately: either revert, or re-record the golden fixtures and record the divergence as a reviewed decision. Silent divergence is the one unacceptable outcome.
4. Listen. The render tests prove the engine is well-behaved, never that it sounds right. Capture an after render and compare.
5. Re-measure if you touched anything on the audio path: CPU, resident memory and allocation count, into `docs/perf-desktop.md`.
6. One change per review. A human reviews every task in this repository.

Stop and ask before you do any of these

- Anything that would make `pgae/` see `pce/` or the reverse — if a change seems to require it, the design is wrong, not the firewall.
- Adding a third-party dependency. The stack is deliberately locked to C++17, CMake, miniaudio and GoogleTest.
- Changing the shape of the C ABI — adding, removing or altering a declaration in `include/prism/prism_core.h`. Additions are a minor version and still need sign-off; changes are a major version.
- Anything that relaxes a real-time rule, even temporarily, even behind a flag.
- Anything requiring network access, or writing raw input events anywhere.
- Adding a module (for example the proposed `pghe/`).

Known sharp edges in the current code

These were found by review and are not yet fixed. If you are working nearby, know about them.

- **Scene ownership across a swap.** The retirement handshake between the audio and control threads has a narrow window in which a stranded scene can be released while still referenced. Read `prism_core.cpp` around the crossfade paths carefully before changing them.
- `crossfade_active()` **reads two atomics** and can therefore report a state that never existed as a single instant. Fine for the UI hint it is used for; not a synchronisation primitive.
- **Denormal performance on x86.** Long decays into near-silence can hit denormal arithmetic and cost roughly an order of magnitude on some x86 targets. Not observed as a dropout, but it is there.
- **The `asan` preset does not link** as-is: the instrumented-allocator test lacks the `nothrow` `new/delete` overloads the sanitizer expects.
- **Stem loop lengths are not coprime** in the shipped asset set: `bed`, `sub` and `air` are 8 s, `lead` 6 s and `pulse` 4 s, so the whole composite repeats every 24 s instead of running for minutes without recurrence. Re-cutting the stems to coprime lengths is an asset change, not a code change.

Part IV — Embedding the engine in your software

The rule that makes everything else simple

Your software talks to exactly one file: `include/prism/prism_core.h`. Nothing else in the repository is public. The desktop harness in this repo is written against that header and nothing else, deliberately, so it doubles as a worked reference. If you find yourself wanting to include a module header, stop — the ABI is missing something and that is a conversation, not a workaround.

The handle is opaque: `prism_core*`. One handle is one complete engine — one inference engine, one audio engine, and the exchange between them. Nothing is global, so multiple handles in one process are fine.

What you build and link

Target	Artifact	How you load it
Desktop (Linux/macOS/Windows)	<code>libprism_core.{a,so,dylib,dll}</code>	Link directly, or <code>dlopen</code> . Set <code>PRISM_BUILD_SHARED=ON</code> for the loadable form.
Android	<code>libprism_core.so</code> per ABI	Bundle in the APK; <code>DynamicLibrary.open</code> from Dart, or JNI from Kotlin.
iOS	<code>prism_core.xcframework</code>	Built by <code>tools/build_ios_framework.sh</code> ; embedded and signed by the app build.
Flutter / Dart	the shared library above	<code>package:prism_core_bindings</code> (ffigen-generated from the header).

Only `prism_*` symbols are exported; everything else is hidden. A shared build exposes exactly the twenty functions in Appendix A.

Lifecycle: the call order that is legal

```
prism_create(&config, &core)           // always first
prism_load_scene(core, "scenes.json")  // decodes every stem HERE. Blocking. Once per handle.
prism_haptic... (not implemented)
prism_start(core)                      // emits the neutral vector, starts the inference thread
  prism_report_* (any thread, any time, repeatedly)
  prism_get_psv (any thread, poll)
  prism_render (ONE audio thread) -- or -- prism_device_start (library owns the device)
  prism_set_mood_override / prism_clear_mood_override
  prism_crossfade_scene / prism_crossfade_active
prism_stop(core)                       // stops inference + device; idempotent; restartable
prism_destroy(core)                   // only after your audio thread has fully quiesced
```

Rule	Consequence if broken
<code>prism_load_scene</code> before <code>prism_start</code> , once per handle	<code>PRISM_ERROR_INVALID_STATE</code> . There is no way to load a second scene; use <code>prism_crossfade_scene</code> to move between scenes.
<code>prism_get_psv</code> only after <code>prism_start</code>	<code>PRISM_ERROR_INVALID_STATE</code> — no vector exists before start.
Pull model and built-in device are mutually exclusive	Calling <code>prism_render</code> while the device runs renders the same stream twice and consumes the state twice.
Quiesce before destroy	Undefined behaviour — use-after-free. See the callout below; this is the crash integrators actually hit.

The one rule that will crash you. No library can stop a thread it does not own. If your audio thread can be inside `prism_render` when `prism_destroy` runs, you have a use-after-free. Stop your audio callback, join or otherwise prove the thread has left the library, and only then destroy. The built-in device and the internal inference thread are owned by the library and are shut down for you.

Threading contract

Thread	May call	Must not
Your control thread	<code>create</code> , <code>load_scene</code> , <code>start</code> , <code>stop</code> , <code>destroy</code> , <code>crossfade_scene</code> , <code>device_*</code>	Call these concurrently with each other — they are not thread-safe against one another.
Any thread	<code>prism_report_*</code> , <code>prism_get_psv</code> , <code>prism_set_mood_override</code> , <code>prism_clear_mood_override</code> , <code>prism_crossfade_active</code>	Call <code>prism_report_*</code> from the audio callback — it takes a lock.
Exactly one audio thread	<code>prism_render</code>	Two threads calling <code>prism_render</code> breaks the single-reader contract on the state exchange.
Library-owned (inference, device)	—	You never see these; <code>prism_destroy</code> shuts them down.

How to pass inputs

Map your platform's signals onto the three report calls. Timestamps are epoch milliseconds from the host clock — use one consistent clock, and do not mix monotonic and wall-clock values.

Platform	Window switch	Idle heartbeat
macOS	NSWorkspace active-application notification	CGEventSourceSecondsSinceLastEventType on a 5 s timer
Windows	SetWinEventHook(EVENT_SYSTEM_FOREGROUND)	GetLastInputInfo on a 5 s timer
Linux/X11	_NET_ACTIVE_WINDOW property change	XScreenSaver idle time on a 5 s timer
Android	App/activity lifecycle or usage-stats transitions	Time since last touch/interaction, on a 5 s timer
iOS	App foreground/background transitions (system-limited)	Time since last interaction within your app

For task deadlines, send the whole list whenever it changes — the call replaces the previous snapshot, so there is no add/remove protocol. An empty list is valid and means 'no deadline pressure, and low confidence because of it'. The list is capped at 4096 entries.

Privacy is your responsibility at the boundary, not the engine's. The ABI cannot receive an app name, so the engine cannot leak one. But your capture layer sees them. Hash and discard in the shell; forward only the bare timestamp. That is what makes the whole-system privacy claim true.

How to receive output

Audio, model 1 — you own the device (recommended for apps)

```
uint32_t rate = prism_sample_rate(core);    // 0 until a scene is loaded
// ... open your own output stream at `rate`, mono float32 ...
// inside your audio callback, on ONE thread:
prism_result r = prism_render(core, out_frames, frame_count);
// r == PRISM_OK and out_frames is filled; on error the buffer is filled with silence.
```

This is what a Flutter app, a game engine or a DAW plugin does. It keeps the device policy — latency, routing, ducking, session category — entirely in your hands, and it is the path the Dart bindings use.

Audio, model 2 — the library owns the device (quick starts, embedded)

```
prism_device_start(core);    // opens a default playback device and renders internally
// ... nothing else to do; do NOT also call prism_render ...
prism_device_stop(core);
```

State read-out, for your UI

```
prism_psv psv;
if (prism_get_psv(core, &psv) == PRISM_OK && psv.sequence != last_seen) {
    last_seen = psv.sequence;
    // psv.arousal, psv.arousal_confidence, ... psv.readiness_confidence
    // psv.update_timestamp_ms, psv.mode_hint (empty string == no hint)
    render_meters(psv);
}
```

Poll on a UI timer — once a second is plenty, since the engine emits at most every few seconds. Deduplicate on sequence, which is strictly monotonic across the handle's life including restarts. Show confidence somewhere: a value without its confidence is misleading, and users notice when a meter moves confidently on no evidence.

Steering the engine from the host: mood override

Sometimes a human must overrule inference — a venue manager picking 'evening warmth', a user choosing a mode. A mood is **a point in state-vector space**, not a volume knob:

```
prism_mood_override o = {0};
o.mode_hint = "venue_peak";
o.arousal = 0.85; o.valence = 0.70; o.cognitive_load = 0.30; o.readiness = 0.80;
o.confidence = 1.0;           // a manual pin is a statement of fact: use 1.0
prism_set_mood_override(core, &o); // published immediately, then ramped like any change
// ...
prism_clear_mood_override(core);    // inference resumes at the next tick
```

Confidence here is load-bearing for the reason Part II demonstrated: publish a mood at confidence 0.3 and the engine will pull it most of the way back to neutral and barely change the sound. The override reaches the audio through the same mapping, smoothing and limiter as an inferred vector — nothing is bypassed.

Changing scenes without a gap

```
prism_scene_swap swap = {0};
swap.scenes_json_path = "scenes_evening.json";
swap.crossfade_ms = 4000;           // 0 selects 1500 ms; clamped to [50, 120000]
swap.align_to_loop_boundary = 1;    // musically clean, but costs up to one loop of delay
prism_result r = prism_crossfade_scene(core, &swap); // BLOCKS to decode; control thread only
if (r == PRISM_ERROR_BUSY) { /* a crossfade is already in flight; retry later */ }
```

The new scene's sample rate must match the loaded one — there is no resampler. Use `prism_crossfade_active` to avoid asking for a swap that would only come back busy.

Error codes and what to do

Code	Means	Your move
PRISM_OK	Success	—
INVALID_ARGUMENT	Null or malformed parameter; out-of-range value; a swap whose sample rate differs	Fix the call. This is a programming error, not a runtime condition.
INVALID_STATE	The call violates the lifecycle — no scene, not started, already started, second scene load	Check your ordering against the lifecycle diagram.
I/O	Scene manifest or a stem file is unreadable or invalid	Check paths: stem paths resolve relative to the manifest, not the working directory.
DEVICE	The built-in device could not be opened or started	Fall back to the pull model, or surface a device-selection UI.
OUT_OF_MEMORY	Decoding the scene exhausted memory	Smaller or fewer stems. Decoded audio is the dominant allocation.
BUSY	A scene crossfade is already in flight	Retry after <code>prism_crossfade_active</code> goes false. Not an error, a state.

`prism_result_description(code)` returns a static human-readable string for logs. No C++ exception ever crosses the ABI; internal failures surface as these codes.

A complete minimal integration

```
#include "prism/prism_core.h"
prism_core* core = NULL;
prism_config cfg = prism_config_default();
cfg.tz_offset_min = -240; // UTC minus local, in minutes (UAE: -240)
if (prism_create(&cfg, &core) != PRISM_OK) return 1;
if (prism_load_scene(core, "assets/scenes.json") != PRISM_OK) { prism_destroy(core); return 1; }
// tell the engine what the user is facing
prism_task_deadline tasks[] = { { now_ms() + 24*3600*1000LL, 2 } }; // due tomorrow, high
prism_report_task_deadlines(core, tasks, 1);
if (prism_start(core) != PRISM_OK) { prism_destroy(core); return 1; }
prism_device_start(core); // simplest audio path
// ---- your 5-second heartbeat, on any thread ----
// prism_report_idle(core, now_ms(), idle_ms_from_os());
// prism_report_app_switch(core, now_ms()); // only when focus actually changes
// ---- your 1-second UI poll ----
// prism_psv psv; prism_get_psv(core, &psv);
prism_device_stop(core);
prism_stop(core);
prism_destroy(core); // audio thread must be quiesced first
```

The same thing in Dart

```
final core = PrismCore.create(tzOffsetMin: -240);
core.loadScene('assets/scenes.json');
core.reportTaskDeadlines([TaskDeadline(dueMs: ..., priority: 2)]);
core.start();
Timer.periodic(const Duration(seconds: 5), (_) {
  core.reportIdle(DateTime.now().millisecondsSinceEpoch, idleMs);
});
Timer.periodic(const Duration(seconds: 1), (_) {
  final psv = core.psv; // null before start
  setState(() => _arousal = psv?.arousal ?? 0.5);
});
```

Integration checklist

- Idle heartbeat on a fixed timer, roughly every 5 s — not only when the user goes idle.
- App-switch events only on real focus changes, with app identity stripped in your shell.
- Full task list re-sent on every change; empty list is legal.
- One consistent epoch-millisecond clock for every timestamp.
- Exactly one audio path chosen: your callback or the built-in device, never both.
- `prism_render` called from exactly one thread.
- Audio thread proven quiesced before `prism_destroy`.
- UI deduplicates on sequence and displays confidence alongside value.
- Every return code checked; BUSY handled as a retry, not a failure.
- Timezone offset set correctly at creation — a wrong offset silently shifts the whole circadian curve.

Things integrators get wrong

- **Treating the engine as request/response.** It is a continuous system. You are not asking a question; you are keeping a model warm. Sparse input produces low confidence, and low confidence produces neutral output — which looks like 'it isn't working'.
- **Expecting instant audible response.** Inference ticks every 5 s and emits every 30 s (or on a 0.1 change); then gains glide over 0.25 s, the filter over 0.6 s, and layer changes wait for a loop boundary. Seconds, by design.
- **Sending idle only when away.** The single most common integration bug. See the heartbeat callout.
- **Re-creating the handle to 'reset'.** Use `prism_stop` then `prism_start`; the sequence continues and the scene stays loaded.
- **Calling `prism_report_*` from the audio callback.** It takes a lock. Use a queue on your side.
- **Assuming stereo.** Output is mono float32; duplicate to both channels yourself.

Where things live in the repository

Path	What is in it	Touch it if...
<code>include/prism/prism_core.h</code>	The only public header. The entire integration surface.	You are integrating. Changing it is a stop-and-ask.
<code>psv/</code>	The contract: vector types, validation, JSON, the real-time snapshot and the wait-free exchange. Depends on nothing.	You are adding a dimension (a major decision) or a transport.
<code>pce/</code>	Inference: the three adapters, fusion, the emit policy. Depends on <code>psv</code> only.	You are changing how state is inferred.
<code>pgae/</code>	Audio: mapping, scene loading, the render engine and DSP. Depends on <code>psv</code> only.	You are changing how state becomes sound.
<code>core/</code>	The composition root and the C ABI implementation. The only place both engines are linked.	You are changing lifecycle, threading or the ABI.
<code>harness/</code>	Desktop harness, written against the C ABI only. The worked integration reference.	You want a runnable example, or an offline render.
<code>bindings/dart/</code>	ffigen-generated bindings plus a hand-written idiomatic wrapper.	You are integrating from Flutter.
<code>examples/flutter_smoke/</code>	Minimal Flutter app: start/stop, live meters, audio on a device.	You want an end-to-end mobile example.
<code>assets/scenes.json</code>	Scene manifest; stem paths resolve relative to it.	You are changing the musical material.
<code>tests/</code>	Unit tests, golden traces, and the firewall and real-time audits.	Always, before and after any change.
<code>docs/</code>	The PSV contract, the PGAE consumption spec, the PGHE proposal, perf records, the build plan.	You want the normative version of anything in this guide.
<code>CLAUDE.md</code>	The repository constitution: invariants, stack lock, stop-and-ask list.	Read it first, once, properly.

If this guide and the documents in `docs/` ever disagree, the documents win: they are the contracts under review, and this guide is an explanation of them.

Appendix A — ABI function reference

Function	Thread	Purpose
<code>prism_version()</code>	any	"MAJOR.MINOR.PATCH" of the linked library; compare against the header macros.
<code>prism_result_description(r)</code>	any	Static human-readable string for a result code.
<code>prism_config_default()</code>	any	Config with all defaults (Aqademiq profile, UTC, spec cadence).
<code>prism_create(cfg, &out)</code>	control	Create an engine. <code>cfg</code> may be NULL.
<code>prism_destroy(core)</code>	control	Stop everything owned by the library and free. NULL is a no-op.
<code>prism_load_scene(core, path)</code>	control	Decode and preload every stem. Blocking I/O happens here.
<code>prism_start(core)</code>	control	Emit the neutral vector; launch the inference thread.
<code>prism_stop(core)</code>	control	Stop inference and the built-in device. Idempotent.
<code>prism_report_app_switch(core, t)</code>	any	One context switch at epoch-ms <code>t</code> .
<code>prism_report_idle(core, t, idle)</code>	any	Idle heartbeat: at <code>t</code> , idle for <code>idle</code> ms.
<code>prism_report_task_deadlines(core, ts, n)</code>	any	Replace the task snapshot. <code>n</code> ≤ 4096.
<code>prism_get_psv(core, &out)</code>	any	Copy the most recent emitted vector.
<code>prism_set_mood_override(core, &o)</code>	any	Pin the vector; published immediately.
<code>prism_clear_mood_override(core)</code>	any	Release the pin; inference resumes next tick.
<code>prism_crossfade_scene(core, &swap)</code>	control	Decode a new scene and arm an equal-power swap.
<code>prism_crossfade_active(core)</code>	any	Non-zero while a swap is armed or running.
<code>prism_sample_rate(core)</code>	any	Sample rate of the loaded scene; 0 before load.
<code>prism_render(core, out, frames)</code>	audio	Render mono float32. Wait-free, allocation-free.
<code>prism_device_start(core)</code>	control	Open and start the built-in playback device.
<code>prism_device_stop(core)</code>	control	Stop and close it. Idempotent.

Appendix B — Constant reference

Constant	Value	Where
Deadline decay tau	48 h	<code>pce/include/pce/deadline.h</code>
Priority weights (low/med/high)	0.4 / 0.7 / 1.0	<code>pce/include/pce/deadline.h</code>
Deadline confidence	0.9 (0 when list empty)	<code>pce/include/pce/deadline.h</code>
Behavioural window	120 s	<code>pce/include/pce/attention.h</code>
Expected heartbeat spacing	5 s (24 samples per window)	<code>pce/include/pce/attention.h</code>
Idle threshold	15 s	<code>pce/include/pce/attention.h</code>
Switch scale	6 per minute	<code>pce/include/pce/attention.h</code>
Behavioural confidence ceiling	0.7 x coverage	<code>pce/include/pce/attention.h</code>
Circadian peak / trough / dip	16:00 / 04:00 / 14:00	<code>pce/src/circadian.cpp</code>
Circadian confidence	0.5 (fixed)	<code>pce/src/circadian.cpp</code>
Fusion importances (arousal)	deadline 1.0, circadian 0.7, behaviour 0.7	<code>pce/src/fusion.cpp</code>

Constant	Value	Where
Fusion importances (load)	deadline 1.0, scatter 0.9	pce/src/fusion.cpp
Fusion importances (readiness)	circadian 1.0, inverse deadline 0.4	pce/src/fusion.cpp
Emit cadence / significant delta	30 s / 0.1	PceOptions, overridable via prism_config
Inference tick	5 s	prism_config.check_interval_ms
Brightness	$0.55 + 0.9a - 0.8l$	pgae/src/mapping.cpp
Cutoff range	300 Hz .. 12 kHz (log)	pgae/src/mapping.cpp
Density	$0.5 + 0.9a - 1.0l$	pgae/src/mapping.cpp
Density gates (pulse/air/lead)	0.35 / 0.55 / 0.72	pgae/src/mapping.cpp
Gain to amplitude	$0.4 \times g^{1.5}$	pgae/src/engine.cpp
Smoothing (gain / cutoff)	0.25 s / 0.6 s	pgae/include/pgae/engine.h
Density crossfade	1.5 s, equal power	pgae/include/pgae/engine.h
Master gain / fade-in	0.7 / 0.5 s	pgae/include/pgae/engine.h
Limiter	-3 dBFS, 3 ms attack, 100 ms release	pgae/include/pgae/engine.h

Appendix C — Glossary

Term	Meaning
PCE	Prism Context Engine. Turns raw events into a state vector. Knows nothing about audio.
PSV	Prism State Vector. Four dimensions, each a value plus its own confidence. The only thing that crosses between the two engines.
PGAE	Prism Generative Audio Engine. Turns a state vector into sound. Knows nothing about inputs.
PGHE	Prism Generative Haptics Engine — specified, not implemented . A second consumer of the same vector.
Effective value	A dimension collapsed with its confidence: $0.5 + (\text{value} - 0.5) \times \text{confidence}$. What actually drives output.
The firewall	The structural rule that the PCE emits only a state vector and the PGAE reads only a state vector, enforced by the build graph and a test.
Scatter	How fragmented attention is, from window-switch frequency. 0 settled, 1 constant switching.
Activity	The fraction of recent idle samples that were below the away threshold. 1 present, 0 away.
Density	A mapped scalar that gates how many stem layers are audible.
Stem	One pre-decoded audio loop with a role: bed, sub, pulse, lead or air.
Scene	A named set of stems in one musical key, described by scenes.json.
Pull model	You own the audio device and call prism_render. The alternative is the built-in device.
Golden trace	A recorded input trace plus the state trajectory the validated probe produced from it; the parity bar is 1e-6.

This document describes commit 2f55a27. When the engine changes, the tables in Part II can be regenerated by rebuilding and re-running the probe that produced them; the invariants in Part III change only by deliberate decision.